Experiment No.:9

Name of Experiment: Create the interactive user input code line in r using readline() function.

Aim:

The readline() function in R is used to get user input interactively. You can prompt the user to input values from the console and store those values in variables for further use.

SOURCE CODE:

```
 Step 1: Use readline() to get user input

 Asking for the user's name
name <- readline(prompt = "Enter your name: ")

 Asking for the user's age
age <- as.numeric(readline(prompt = "Enter your age: "))

 Asking for the user's favorite number
favorite_number <- as.numeric(readline(prompt = "Enter your favorite number: "))

 Step 2: Display the inputs back to the user
cat("\nHello,", name, "! You are", age, "years old, and your favorite number is", favorite_number,
".\n")

 Step 3: Simple logic based on input
if (favorite_number %% 2 == 0) {
  cat("Your favorite number is even.\n")
} else {
  cat("Your favorite number is odd.\n")
}
```

OUTPUT:

```
Enter your name: Alice

Enter your age: 25

Enter your favorite number: 42


Hello, Alice! You are 25 years old, and your favorite number is 42.

Your favorite number is even.
```

Basic Level Questions

1. Which function in R is used to get user input interactively?
   Answer: readline()
2. What does **as.numeric()** do in this experiment?
   Answer: Converts input from string to numeric
3. Which operator is used to check if a number is even or odd?
   Answer: %% (modulus operator)
4. Which function is used to display output in a formatted manner?
   Answer: cat()
5. What type of data does **readline()** return by default?
   Answer: Character (string)

Moderate Level Questions

1. Write the R code to ask for a user's city using **readline()**.
   Answer: city <- readline(prompt = "Enter your city: ")
2. How would you check if the user's age is above 18 and print "Adult"?
   Answer:

```
if (age > 18) {
  cat("Adult\n")
}
```

3. What will be the result of 25 %% 4?
   Answer: 1
4. How do you convert user input to integer type (with rounding)?
   Answer: as.integer(readline())
5. Write a line of code to print the square of favorite number.
   Answer: cat(favorite_number^2)

Advanced Level Questions

1. How can you ensure the user enters a numeric value and re-prompt if not?
   Answer:

```
repeat {
  input <- readline("Enter a number: ")
  if (!is.na(as.numeric(input))) break
  cat("Please enter a valid number.\n")
}
```

2. Write a function that takes name and age from user and returns a greeting.
Answer:

```
greet_user <- function() {
  name <- readline("Name: ")
  age <- as.numeric(readline("Age: "))
  return(paste("Hello", name, "You are", age, "years old."))
}
```

3. How would you check if both name and favorite number are valid before continuing?
Answer:

```
if (nzchar(name) && !is.na(favorite_number)) {
  cat("Valid inputs.\n")
}
```

4. Add code to count the number of vowels in the user's name.
Answer:

```
vowels <- sum(tolower(strsplit(name, "")[[1]]) %in% c("a", "e", "i", "o", "u"))
cat("Number of vowels in name:", vowels, "\n")
```

5. How to store multiple user entries in a loop for 3 students (name & age)?
Answer:

```
names <- c()
ages <- c()
for (i in 1:3) {
  names[i] <- readline(paste("Enter name", i, ": "))
  ages[i] <- as.numeric(readline(paste("Enter age", i, ": ")))
}
```

Use Cases:

- Interactive Data Collection in Console Applications
- Command-Line-Based Survey Tools
- Dynamic Input Validation System
- Real-Time User Decision Branching
- Educational Practice Programs for Beginners

Experiment No.:10

Name of Experiment: Create the contingency table for the given vector format data.

Aim: To understand the process of summarizing categorical data using a contingency table in R

SOURCE CODE:

Suppose we have two vectors:

1. Gender: Represents the gender of a group of students.
2. Study Method: Represents the preferred study method (Online or In-person).

```
Vector for Gender (categorical variable 1)
gender <- c("Male", "Female", "Male", "Female", "Male", "Female", "Male", "Female")

Vector for Study Method (categorical variable 2)
study_method <- c("Online", "In-person", "In-person", "Online", "Online", "Online", "In-person",
"In-person")
```

R Code to Create a Contingency Table

```
Step 1: Create the contingency table
contingency_table <- table(gender, study_method)

Step 2: Display the contingency table
print(contingency_table)
```

OUTPUT:

```
            study_method
gender    In-person Online
Female        2      2
Male          2      2
```

Basic Level Questions

1. Which R function is used to create a contingency table from two vectors?
   Answer: `table()`
2. What type of data does **`table()`** summarize?
   Answer: Categorical data
3. How do you print a table in R?
   Answer: `print()` or just typing the table name
4. How many arguments does the **`table()`** function require at minimum?
   Answer: Two (for bivariate table), one for univariate
5. What does the output of a contingency table represent?
   Answer: Frequencies of combinations of categories

Moderate Level Questions

1. **Write the R code to find how many females preferred "In-person".**
   Answer: `contingency_table["Female", "In-person"]`
2. **How do you get total number of students from the table?**
   Answer: `sum(contingency_table)`
3. **Write code to create a proportion table from the contingency table.**
   Answer: `prop.table(contingency_table)`
4. **How can you convert the contingency table into a data frame?**
   Answer: `as.data.frame(contingency_table)`
5. **Which function would you use to summarize row-wise totals?**
   Answer: `rowSums(contingency_table)`

Advanced Level Questions

1. Write R code to perform a chi-square test on the contingency table.
   Answer: `chisq.test(contingency_table)`
2. Write code to filter only rows where Online preference is ≥ 2.
   Answer:

```
subset(as.data.frame(contingency_table), Online >= 2)
```

3. How would you visualize this contingency table using base R?
   Answer: `barplot(contingency_table, beside=TRUE, legend=TRUE)`
4. Write code to normalize the table column-wise.
   Answer: `prop.table(contingency_table, 2)`
5. How can you transpose the contingency table?
   Answer: `t(contingency_table)`

Use Cases:

- Frequency Analysis of Survey Data
- Cross-Tabulation for Demographic Insights
- Summary of Preference Patterns by Category
- Input Validation in Categorical Datasets
- Data Preparation for Chi-Square Test

Experiment No.:11

Name of Experiment: Convert the contingency table to original format of the given data.

Aim: To reconstruct the original dataset from a summarized contingency table.

SOURCE CODE:

Example Contingency Table

Let's use the contingency table from the earlier example:

```
          study_method
gender    In-person Online
Female          2      2
Male            2      2
```

Goal:

We will reverse this contingency table back into the original format of two vectors

R Code: Converting a Contingency Table Back to Original Data

```
Step 1: Create the original contingency table
contingency_table <- matrix(c(2, 2, 2, 2), nrow = 2, byrow = TRUE)
rownames(contingency_table) <- c("Female", "Male")
colnames(contingency_table) <- c("In-person", "Online")

Step 2: Convert the contingency table back to original data

Expanding the table to individual observations
gender <- rep(rownames(contingency_table), times = rowSums(contingency_table))
study_method <- rep(colnames(contingency_table), each=nrow(contingency_table), times =
contingency_table)

Step 3: Combine into a data frame
original_data <- data.frame(Gender = gender, Study_Method = study_method)

Step 4: Display the original data
print(original_data)
```

Output:

This will reconstruct the original dataset as:

```
  Gender Study_Method
1 Female    In-person
2 Female    In-person
3 Female       Online
4 Female       Online
5   Male    In-person
6   Male    In-person
7   Male       Online
8   Male       Online
```

Explanation:

- The contingency table provided the counts of how many times each combination (e.g., "Female-In-person", "Male-Online") occurred.
- The rep() function is used to replicate the category labels (e.g., "Male", "Female", "In-person", "Online") based on the counts in the contingency table.
- This results in a reconstructed dataset in its original raw format.

Basic Level Questions

1. Which function is used to replicate values in R?
   Answer: rep()
2. What is the purpose of **rownames()** in R?
   Answer: To assign or retrieve row names of a matrix or data frame.
3. What data structure does **table()** output?
   Answer: A contingency table (matrix with named rows and columns).
4. Which function combines vectors into a data frame?
   Answer: data.frame()
5. Which R function is used to print values to the console?
   Answer: print()

Moderate Level Questions

1. How do you get the total number of rows to reconstruct from a contingency table?
   Answer: sum(contingency_table)
2. Write R code to extract the row totals from a contingency table.
   Answer: rowSums(contingency_table)
3. How do you assign column names to a matrix in R?
   Answer: colnames(matrix_name) <- c(...)
4. What does **each =** in **rep()** signify?
   Answer: Number of times each element of the input is repeated.
5. Why is **times =** used in **rep()** for this task?
   Answer: To replicate elements according to counts from the contingency table.

Advanced Level Questions

1. Write code to convert any contingency table to a long-format data frame.
Answer:

```
as.data.frame(as.table(contingency_table))
```

2. How would you validate that the reconstructed data has correct frequency counts?
Answer:

```
table(original_data$Gender, original_data$Study_Method) == contingency_table
```

3. **Write code to sort the reconstructed data alphabetically by gender.**
   Answer:

```
original_data <- original_data[order(original_data$Gender), ]
```

4. **Modify code to include a third categorical variable like "Course".**
   Answer:

   ```
   original_data$Course <- rep("Statistics", nrow(original_data))
   ```

5. **Write a function to generalize the reconstruction of data from any 2D contingency table.**
   Answer:

   ```
   reconstruct_data <- function(tab) {
     gender <- rep(rownames(tab), times = rowSums(tab))
     study_method <- rep(colnames(tab), each = nrow(tab), times = tab)
     data.frame(Gender = gender, Study_Method = study_method)
   }
   ```

Use Cases:

- Data Reconstruction from Summary Tables
- Reversing Cross-tabulated Data
- Raw Data Recovery in Categorical Analysis
- Simulation of Survey Responses from Frequency Tables
- Testing Backward Data Conversion Techniques

Experiment No.:12

Name of Experiment: Analyse and give interpretation of summary statistics for the given data.

Aim: To calculate and interpret key summary statistics for a given set of data, enabling us to understand the central tendency, spread, and overall distribution of the data.

SOURCE CODE:

Example Data:

```
scores <- c(85, 90, 88, 75, 95, 70, 78, 85, 80, 85)
```

R Code to Calculate Summary Statistics:

```
 Step 1: Input the data
scores <- c(85, 90, 88, 75, 95, 70, 78, 85, 80, 85)

 Step 2: Calculate the summary statistics
summary_stats <- summary(scores)
sd_scores <- sd(scores)   Standard deviation

 Step 3: Display the summary statistics and standard deviation
print("Summary Statistics:")
print(summary_stats)
cat("\nStandard Deviation:", sd_scores, "\n")
```

OUTPUT:

```
 Summary Statistics:
 Min. 1st Qu.  Median    Mean 3rd Qu.    Max.
 70.00   78.75   85.00   83.10   88.75   95.00

Standard Deviation: 7.671066
```

Interpretation of the Summary Statistics:

1. **Minimum**: The lowest score in the dataset is **70**.
2. **1st Quartile (Q1)**: The first quartile (25th percentile) is **78.75**, meaning that 25% of the students scored less than or equal to this value.
3. **Median (Q2)**: The median (50th percentile) is **85**, indicating that half the students scored below and half above this value.
4. **Mean (Average)**: The average score is **83.1**. This is the central tendency of the dataset.

5. **3rd Quartile (Q3)**: The third quartile (75th percentile) is **88.75**, meaning that 75% of the students scored less than or equal to this value.
6. **Maximum**: The highest score in the dataset is **95**.

7. **Standard Deviation**: The standard deviation is **7.67**, showing the spread of the scores around the mean. A higher standard deviation indicates more variability in scores.

Basic Level Questions

1. Which function is used to get summary statistics in R?
   Answer: summary()
2. What function is used to calculate the standard deviation in R?
   Answer: sd()
3. What does the **median()** function return?
   Answer: The middle value (50th percentile) of the data.
4. What is the output of **mean(c(10, 20, 30))**?
   Answer: 20
5. What is the purpose of **cat()** function in R?
   Answer: To concatenate and print output to the console.

Moderate Level Questions

1. Write R code to find the mean score.
   Answer: mean(scores)
2. How do you interpret the 1st quartile from summary statistics?
   Answer: 25% of the data falls below or equals the 1st quartile.
3. Write code to extract the maximum score from summary.
   Answer: max(scores)
4. What does a standard deviation of 7.67 indicate about score distribution?
   Answer: There is moderate spread around the mean; scores vary by ~7.67 units.
5. Write code to count how many scores are above the mean.
   Answer: sum(scores > mean(scores))

Advanced Level Questions

1. Write R code to calculate interquartile range (IQR).
   Answer: IQR(scores) or quantile(scores, 0.75) - quantile(scores, 0.25)
2. How can you visualize the distribution of scores?
   Answer: boxplot(scores)
3. How do you detect outliers using summary statistics?
   Answer: Values below Q1 - 1.5*IQR or above Q3 + 1.5*IQR are outliers.
4. Write code to normalize scores using z-score standardization.
   Answer: (scores - mean(scores)) / sd(scores)
5. Create a function that returns summary statistics and standard deviation.
   Answer:

```
get_summary <- function(data) {
  list(Summary = summary(data), SD = sd(data))
}
```

Use Cases:

- Student Score Distribution Analysis
- Sales Performance Summary
- Survey Rating Variability Check
- Exam Result Variance Analysis
- Quality Control in Manufacturing Scores